

Exercício 1: Calculadora de Descontos

Objetivo: Desenvolver uma função que calcule o valor final de um produto após a aplicação de um desconto.

Regras de Negócio:

1. A função deve receber o `valor_original` do produto e o `percentual_desconto`.
2. O `percentual_desconto` deve ser um número entre 0 e 100.
3. Se o `percentual_desconto` for maior que 100 ou menor que 0, deve-se lançar uma exceção (ex: `ValueError`).
4. O valor final deve ser o `valor_original` menos o desconto aplicado.
5. O valor final não pode ser negativo (se o desconto for maior que o valor original, o valor final deve ser 0).

Sugestão de TDD:

- Comece testando um cenário básico de desconto (ex: 10% de desconto em `R$100`).
- Em seguida, adicione testes para os casos de exceção (percentual inválido).
- Por fim, teste o cenário onde o desconto é maior que o valor original.

Exercício 2: Conversor de Temperatura

Objetivo: Criar um módulo com funções para converter temperaturas entre Celsius e Fahrenheit.

Regras de Negócio:

1. A função `celsius_para_fahrenheit` deve converter uma temperatura de Celsius para Fahrenheit.
 - Fórmula: $F = C * 9/5 + 32$
2. A função `fahrenheit_para_celsius` deve converter uma temperatura de Fahrenheit para Celsius.
 - Fórmula: $C = (F - 32) * 5/9$
3. Ambas as funções devem aceitar apenas entradas numéricas. Se uma entrada não numérica for fornecida, deve-se lançar uma exceção (ex: `TypeError`).
4. As funções devem retornar valores com precisão de duas casas decimais.

Sugestão de TDD:

- Comece com um teste simples para `celsius_para_fahrenheit` (ex: 0°C para 32°F).
- Adicione testes para outros valores conhecidos.
- Implemente a função `fahrenheit_para_celsius` seguindo o mesmo padrão.

- Por último, adicione testes para as entradas não numéricas e implemente o tratamento de exceções.

Exercício 3: Sistema de Estacionamento

Objetivo: Desenvolver uma função que calcule o valor a ser pago por um veículo em um estacionamento, com base no tempo de permanência.

Regras de Negócio:

1. A função `calcular_valor_estacionamento` deve receber o `tempo_em_minutos` como entrada.
2. A primeira hora (até 60 minutos) custa R\$ 10,00.
3. Cada hora adicional (ou fração de hora) custa R\$ 5,00.
4. O valor máximo a ser cobrado por um período de 24 horas é R\$ 50,00.
5. Se o `tempo_em_minutos` for zero ou negativo, deve-se lançar uma exceção (ex: `ValueError`).
6. A função deve retornar o valor total a ser pago.

Sugestão de TDD:

- Comece com o caso mais simples: tempo menor ou igual a 60 minutos.
- Adicione um teste para um tempo um pouco maior que 60 minutos (ex: 70 minutos).
- Teste múltiplos períodos de horas adicionais.
- Implemente o limite máximo de 24 horas (R\$ 50,00).
- Por fim, adicione testes para entradas inválidas de tempo.